

Task 1: Data Cleaning & Preprocessing Report

Objective	Target Dataset	Core Tools
Prepare Raw Data for Machine Learning	Titanic: Machine Learning from Disaster	Python, Pandas, NumPy, Scikit-Learn, Seaborn

1. Project Overview & Task Objectives

Machine learning models do not learn strictly from raw real-world data; they learn from mathematical representations of features. Real-world datasets consistently exhibit severe imperfections including missing values, noisy text representations, skewed distributions, extreme outliers, non-standardized feature scales, and unencoded categorical variables. The primary objective of this project is to implement a production-grade data cleaning and preprocessing pipeline on the canonical Titanic dataset (891 passenger records across 12 initial attributes).

2. Dataset Exploration & Structural Assessment

The raw Titanic dataset represents passenger demographics, booking tickets, cabin locations, and survival outcomes. Initial examination revealed significant structural challenges:

Feature	Data Type	Missing Count	Preprocessing Challenge & Strategy
passengerid / ticket	Int / Object	0 (0.0%)	High cardinality unique identifiers with no predictive power; dropped.
survived	Int64	0 (0.0%)	Binary ground truth target variable (0 = Died, 1 = Survived); separated prior to transformations.
age	Float64	177 (19.8%)	Missing values present; imputed using robust median strategy and capped against outliers.
fare	Float64	0 (0.0%)	Extreme positive skewness and high outlier range; clipped using IQR thresholds and standardized.
cabin / deck	Object	687 (77.1%)	Massive missingness; extracted first letter representing deck layer; imputed missing as

			'Unknown'.
embarked / sex	Object	2 / 0	Categorical variables; embarked imputed with mode ('S'), sex and embarked one-hot encoded.

3. Step-by-Step Data Preprocessing Pipeline

The end-to-end data pipeline integrates automated data cleaning, domain-informed feature engineering, exploratory data visualization, robust outlier remediation, and Scikit-Learn Pipeline transformers:

Step 1: Ingestion and Header Sanitization

Raw headers often contain unpredictable capitalization, non-standard special characters, or leading spaces. We sanitize column names into clean, lowercased snake_case identifiers (`df.columns.str.lower().str.replace(r'^a-z0-9+', '_')`).

Step 2: Feature Engineering (Passenger Title, Deck, & Family Size)

Domain-specific feature extraction uncovers latent signals that significantly enhance model discriminative power:

- **Social Title (title):** Extracted from passenger names (e.g., 'Mr', 'Mrs', 'Miss', 'Master'). Infrequent nobility/officer titles ('Don', 'Rev', 'Dr', 'Col', 'Countess') are mapped to an 'Other' group to avoid excessive high-cardinality dimensionality.
- **Deck Layer (deck):** Extracted from the initial letter of the cabin identifier, delineating vertical location on the vessel (Deck A through G).
- **Family Size (family_size):** Computed as `sibsp (siblings/spouses) + parch (parents/children) + 1 (self)`. This captures household travel dynamics.

Step 3: Visual Outlier Detection & IQR Capping

Rather than arbitrarily deleting rows containing outliers—which causes severe sample loss and survival selection bias—we utilize Tukey's Interquartile Range (IQR) bounding method:

Step 4: Scikit-Learn Pipeline Architecture

To prevent data leakage during cross-validation, preprocessing steps are encapsulated inside modular Scikit-Learn Pipelines orchestrated by a `ColumnTransformer`:

- **Numeric Pipeline:** Imputation using `SimpleImputer(strategy='median')` followed by standard z-score scaling via `StandardScaler()`.
- **Categorical Pipeline:** Imputation using `SimpleImputer(strategy='most_frequent')` followed by `OneHotEncoder(handle_unknown='ignore', sparse_output=False)`.

4. Complete Source Code Implementation

Below is the self-contained, reproducible Python code implementing the end-to-end data cleaning and ML transformation workflow:

```
import re
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Directory Setup & Data Ingestion
Path("data/processed").mkdir(parents=True, exist_ok=True)
Path("screenshots").mkdir(parents=True, exist_ok=True)

df = pd.read_csv("Titanic-Dataset.csv")

# 2. Standardize Column Headers
df.columns = (
    df.columns.astype(str)
    .str.strip()
    .str.lower()
    .str.replace(r"^[a-z0-9]+", "_", regex=True)
    .str.strip("_")
)

# 3. Deduplication & Text Sanitation
df = df.drop_duplicates().copy()
for col in df.select_dtypes(include=["object", "string"]).columns:
    df[col] = df[col].map(lambda x: x.strip() if isinstance(x, str) else x)
    df[col] = df[col].replace("", np.nan)

# 4. Feature Engineering
if "name" in df.columns:
    df["title"] = df["name"].str.extract(r",\s*([^\.]+)\.", expand=False).str.strip()
    common_titles = ["Mr", "Mrs", "Miss", "Master"]
    df.loc[df["title"].notna() & ~df["title"].isin(common_titles), "title"] = "Other"

if "cabin" in df.columns:
    df["deck"] = df["cabin"].astype(str).str[0]
    df["deck"] = df["deck"].replace({"n": np.nan, "N": np.nan, "nan": np.nan})

if {"sibsp", "parch"}.issubset(df.columns):
    df["family_size"] = df["sibsp"] + df["parch"] + 1

# 5. Outlier Detection & IQR Bounding
outlier_cols = [c for c in ["age", "fare"] if c in df.columns]
for col in outlier_cols:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    if pd.notna(iqr) and iqr > 0:
```

```

        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        df[col] = df[col].clip(lower, upper)

# Export intermediate cleaned human-readable dataset
df.to_csv("data/processed/cleaned_titanic_dataset.csv", index=False)

# 6. Feature Separation
target = "survived"
y = df[target].copy() if target in df.columns else None
features = df.drop(columns=[target], errors="ignore").copy()
features.drop(columns=["passengerid", "name", "ticket", "cabin"], errors="ignore", inplace=True)

# 7. Pipeline Construction
num_cols = ["pclass", "age", "sibsp", "parch", "fare", "family_size"]
cat_cols = ["sex", "embarked", "title", "deck"]

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[("num", num_pipe, num_cols), ("cat", cat_pipe, cat_cols)],
    remainder="drop"
)

# 8. Transform & Export ML-Ready Features
X_proc = preprocessor.fit_transform(features)
feat_names = preprocessor.get_feature_names_out()
processed_df = pd.DataFrame(X_proc, columns=feat_names, index=features.index)

if y is not None:
    processed_df["survived"] = y.values

processed_df.to_csv("data/processed/preprocessed_ml_features.csv", index=False)
print("Pipeline executed successfully. Processed shape:", processed_df.shape)

```

5. Interview Questions

1. What are the different types of missing data?

Missing data is formally classified into three fundamental theoretical mechanisms (Rubin's Missing Data Framework):

a) Missing Completely at Random (MCAR): The probability of data missing is entirely independent of observed and unobserved data. Example: A sensor battery dies randomly, or a survey sheet is dropped by accident. Dropping MCAR records typically introduces no systematic bias, though it reduces statistical power.

b) Missing at Random (MAR): The probability of missingness depends systematically on other observed variables, but not on the missing value itself. Example: In the Titanic dataset, men rarely report certain attributes compared to women, but within the observed demographic subgroup, missingness is random. Imputing based on observed conditional subgroups (e.g., median age by Pclass and Sex) completely corrects MAR.

c) Missing Not at Random (MNAR): Missingness directly relates to the unobserved value itself. Example: Wealthy individuals omitting their net worth, or sick patients dropping out of a clinical drug study due to extreme adverse effects. MNAR requires domain modeling, missing indicator indicators (MissingIndicator flag), or pattern-mixture models.

2. How do you handle categorical variables?

Categorical variables cannot be processed directly by algorithms relying on vector arithmetic. Common handling methods include:

- One-Hot Encoding: Creates binary columns for each unique category. Best for nominal data without natural hierarchy (e.g., Embarked: C, Q, S). To prevent linear dependence in linear/logistic regression, one category must be dropped (dummy variable trap, drop='first').
- Ordinal / Label Encoding: Assigns integers representing strict ordinal rank (e.g., Low=0, Medium=1, High=2; Pclass: 1 > 2 > 3). Must never be applied to unordered nominal data, as distance-based algorithms infer false numerical ranking (e.g., Blue=1, Green=2 implies Green > Blue).
- Target / Mean Encoding: Replaces categories with the expected target value (e.g., historical survival rate per Title). Powerful for high cardinality, but prone to data leakage and overfitting; must use K-Fold cross-validation and smoothing regularization (m-estimate).
- Frequency / Count Encoding: Replaces category names with their sample occurrence frequency.
- Feature Hashing / Binary Encoding: Compresses thousands of distinct categories into a fixed-width vector hash.

3. What is the difference between normalization and standardization?

Both techniques rescale numerical features to prevent large-magnitude variables from dominating smaller-magnitude variables:

- Standardization (Z-score Scaling):

Formula: $z = (x - \mu) / \sigma$

Properties: Centers data around mean $\mu = 0$ with standard deviation $\sigma = 1$. Does not bound features into a fixed range. Handles outliers far better because it does not compress dense inliers into a tiny fractional window. Highly recommended for Linear/Logistic Regression, Support Vector Machines (SVM), Principal Component Analysis (PCA), and Neural Networks.

- Normalization (Min-Max Scaling):

Formula: $x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$

Properties: Scales all values strictly into [0, 1] (or [-1, 1]). Highly sensitive to extreme outliers; an anomalous maximum causes all normal records to cluster tightly near 0. Ideal for bounded algorithms (image pixel values [0, 255], K-Nearest Neighbors, Neural Networks with bounded activation functions like Sigmoid).

4. How do you detect and handle outliers?

Outlier detection balances mathematical detection with domain knowledge:

Detection Techniques:

1. Parametric (Z-Score): Computes $z = |(x - \mu) / \sigma|$. A threshold of $|z| > 3$ flags values exceeding 3 standard deviations under normal distributions.
2. Non-Parametric (IQR Rule): $IQR = Q3 - Q1$. Values below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$ are flagged. Immune to skewed non-Gaussian distributions.
3. Multivariate / Machine Learning: Isolation Forests, Local Outlier Factor (LOF), and Elliptic Envelope (Mahalanobis Distance) detect anomalous multi-dimensional combinations.

Handling Strategies:

- Winsorization / Capping (Clipping): Replaces extreme values with percentile thresholds (e.g., 1st and 99th percentiles, or IQR bounds). Preserves sample size.
- Mathematical Transformation: Logarithmic ($\log_{10}$) or Box-Cox transformations compress long-tailed distributions into bell curves.
- Pruning / Dropping: Remove rows only if verified as data corruption or sensor failures; never discard legitimate extreme events in real-world distributions.

5. Why is preprocessing important in Machine Learning?

Machine learning algorithms operate on strict mathematical assumptions:

- Overcoming Numerical Instability: Gradient descent updates step in proportion to feature scales. Unscaled features cause chaotic zig-zagging oscillations, slow convergence, or numerical overflow/exploding gradients.
- Distance Distortion: Metric-based models (KNN, K-Means clustering, SVMs) rely on Euclidean distance. A feature with range 0–100,000 completely drowns out a binary feature (0 or 1), making the algorithm blind to demographic variables.
- Elimination of Data Leakage: Proper preprocessing (fitting encoders and scalers strictly on training folds and only transforming validation/test folds) ensures realistic generalization.
- Signal Amplification: Handling missing values, cleaning corrupted strings, and extracting meaningful interaction terms boosts model signal-to-noise ratio.

6. What is One-Hot Encoding vs. Label Encoding?

- One-Hot Encoding:

- Implementation: Converts 1 categorical column of K distinct levels into K separate binary columns (0 or 1).
- Pros: Introduces no artificial ordinal relationships. Non-parametric.
- Cons: Causes the 'curse of dimensionality' if cardinality is high (e.g., ZIP codes, medical ICD-10 codes), yielding sparse memory bloat.
- Dummy Variable Trap: In regression models with an intercept, having K categories causes perfect multicollinearity (K columns sum to 1). We drop 1 column (drop='first').

- Label Encoding:

- Implementation: Assigns an arbitrary integer (0, 1, 2, ..., K-1) to each category.
- Pros: Memory efficient, retains 1 column.
- Cons: Imposes a linear distance metric (e.g., Dog=0, Cat=1, Elephant=2 incorrectly asserts Elephant is twice Cat and

further from Dog). Tree-based models (XGBoost, Random Forest) can split on label encoded features, but linear models, SVMs, and neural networks misinterpret the values.

- Proper Use: Label encoding is designed strictly for encoding the target label (y), whereas OrdinalEncoder is used for features.

7. How do you handle data imbalance?

Class imbalance occurs when the majority class heavily outnumbers the minority class (e.g., Fraud Detection: 99.8% legit vs 0.2% fraud; Titanic: 61.6% deceased vs 38.4% survived). Standard models optimize accuracy by predicting the majority class for all instances.

Techniques to address imbalance:

1. Resampling Strategies:

- Oversampling: Randomly duplicating minority instances or synthesizing synthetic examples via SMOTE (Synthetic Minority Over-sampling Technique) or ADASYN.
- Undersampling: Randomly removing majority class examples or using cluster centroids / Tomek Links to balance class distribution.

2. Cost-Sensitive Learning:

Adjusting algorithm loss functions to penalize misclassifications of the minority class more heavily (e.g., `class_weight='balanced'` in Scikit-Learn Logistic Regression and Random Forest, or `scale_pos_weight` in XGBoost).

3. Evaluation Metric Selection:

Abandoning accuracy in favor of Precision, Recall, F1-Score, Balanced Accuracy, and Area Under the Precision-Recall Curve (PR-AUC).

4. Anomaly Detection / Threshold Tuning:

Modeling the problem as one-class anomaly detection (Isolation Forests) or lowering the classification decision threshold from 0.5 to a lower value.

8. Can preprocessing affect model accuracy?

Yes, preprocessing can drastically make or break model accuracy. Its impacts include:

• Positive Impact (Performance Gain):

- Robust imputation and clipping prevent outlier distortion, stabilizing linear decision boundaries.
- Standard scaling accelerates gradient convergence in neural networks and linear models from days to seconds.
- Feature extraction (e.g., Passenger Title, Deck) reveals nonlinear relationships that simpler linear algorithms could never discover on raw text.

• Negative Impact (Performance Destruction & Data Leakage):

- Data Leakage: Computing mean/std or fitting an imputer across the entire dataset before train-test split inflates test metrics artificially; when deployed, model accuracy collapses.
- Over-imputation: Imputing a feature with 95% missingness adds synthetic noise that degrades tree splits.
- Dimensionality Explosion: One-hot encoding high-cardinality features causes sparse matrices, leading to severe overfitting in tree-based algorithms.
- Information Loss: Unnecessary deletion of rows containing missing entries discards informative patterns and introduces survival selection bias.

6. Summary Pipeline Architecture

Phase	Standard Industry Practice	Common Pitfall to Avoid
Exploration	Inspect dtypes, null frequencies, and duplicate IDs	Assuming data types are correct without validation
Imputation	Use median for skewed numbers; mode for categories	Using mean imputation on highly skewed distributions
Outliers	Clip with IQR thresholds or transform logarithmically	Dropping real rows without empirical justification
Encoding	One-hot encode nominal; ordinal encode ranked categories	Label encoding nominal variables for linear models
Pipelines	Wrap in ColumnTransformer & fit strictly on train fold	Scaling entire dataset prior to train/test split